

Entrega de la Solución de Software Funcional

Sistema de Asistencia Inteligente de Admisiones: "Nova OS '97"

Evidencia de Producto 3 — Norma SENA 220501096

- **Programa de Formación:** Análisis y Desarrollo de Software (ADSO)
 - **Norma de Competencia:** 220501096 — *Desarrollar solución de software de acuerdo con especificaciones de diseño y marcos de referencia.*
 - **Candidato / Aprendiz:** Brandon Jose Carranza Rangel
 - **Documento de Identidad:** C.C. 1007892884
 - **Organización Beneficiaria:** Nova Idiomas Colombia
 - **Fecha de Elaboración:** 2026-09-02 (Zona Horaria: America/Bogota)
 - **Estado de la Solución:** 100% Funcional, Probada y Desplegable
-

1. Identificación y Acceso al Repositorio del Proyecto

La solución de software completa se encuentra versionada bajo control de versiones Git y alojada en el siguiente repositorio público/privado de GitHub:

- **URL Oficial del Repositorio:** https://github.com/nastex123/NovaVice_os97.git
 - **Rama Principal de Producción:** `main` (Versión etiquetada `v2.6.0`)
 - **Licencia:** MIT License
 - **Lenguajes Predominantes:** Python (65%), TypeScript / React (30%), SQL y Shell Scripts (5%).
-

2. Inventario de Componentes y Código Fuente Entregado

El proyecto se entrega con una estructura modular monorepo limpia y reproducible:

```

NovaVice_os97/
├── backend/                                # Servidor de Inferencia RAG y API REST
│   ├── data/
│   │   ├── documents/                    # 83 Documentos Oficiales Institucionales (Markdown)
│   │   ├── chroma_db/                   # Vector Store persistente con embeddings locales
│   │   └── escalations.json              # Almacén estructurado de tickets humanos
│   ├── src/
│   │   ├── api/                          # Endpoints REST y esquemas Pydantic
│   │   ├── core/                         # Guardrails, navegación, caché y dispatcher
│   │   └── rag/                           # PureBM25, HybridRetriever y RAG Engine
│   ├── tests/                            # 27 Pruebas Automatizadas en Pytest (100% Pass)
│   └── requirements.txt                   # Dependencias de producción Python
├── frontend/                              # Aplicación Web React / Next.js 15
│   ├── src/
│   │   ├── app/                          # App Router (page.tsx, layout.tsx, globals.css)
│   │   ├── components/                   # Componentes UI (Chat, PixiJS, Header, Metrics)
│   │   └── lib/                           # Clientes API fetch y tipos TypeScript
│   ├── package.json                       # Dependencias Node.js
│   └── tailwind.config.ts                 # Configuración de estilos retro synthwave
├── SENA/
│   ├── README.md                         # Evidencias Formativas de Certificación
│   ├── part1/                            # Ficha maestra institucional
│   └── part2/                            # Norma 220501095 (Diseño de Software)
│                                         # Norma 220501096 (Desarrollo de Software)
├── Dockerfile                            # Manifiesto para despliegue en contenedores
├── installer.py / install.bat / install.sh # Suite de instalación de un solo clic
└── run.py / start.bat / start.sh          # Supervisor multi-proceso sincronizado

```

3. Integración con Base de Datos y Persistencia

La solución articula un esquema híbrido de almacenamiento relacional y vectorial:

1. Almacén Vectorial Embebido (ChromaDB):

- Ubicación: `backend/data/chroma_db/`.
- Modelo de Embeddings: `a11-MiniLM-L6-v2` ejecutado en CPU mediante ONNX Runtime.
- Contenido: 252 fragmentos documentales normalizados correspondientes a los 83 reglamentos y tarifas oficiales.

2. Esquema Relacional Estructurado (PostgreSQL / ANSI SQL):

- Script DDL completo disponible en: [SENA/part1/04_MODELO_BASE_DATOS.md](#).
- Entidades normalizadas en 3FN: `aspirantes`, `programas_idiomas`, `sedes_modalidades`, `tarifas_descuentos`, `citas_placement_test` y `tickets_escalamiento`.

3. Persistencia Transaccional de Escalamiento:

- Archivo: `backend/data/escalations.json`.
- Formato de ticket estructurado:

```
{
  "ticket_id": "ESC-20260902-8F12",
  "query": "tienen convenio con la embajada de Australia para visa de trabajo?",
  "user_id": "aspirante_web_104",
  "confidence_score": 0.22,
  "reason": "Out of Scope - Relevancia baja",
  "status": "pending",
  "created_at": "2026-09-02T10:48:00-05:00"
}
```

4. Validación de Calidad y Suite de Pruebas Automatizadas

La aplicación cuenta con una batería de **27 pruebas automatizadas** en `pytest` que verifican la integridad funcional, seguridad contra prompt injections, precisión de lematización léxica y contratos de API:

```
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.2, pluggy-1.5.0
rootdir: c:\Users\Usuario\Documents\GitHub\NovaVice_os97
collected 27 items

backend/tests/test_api_routes.py ..... [ 22%]
backend/tests/test_executables.py ... [ 33%]
backend/tests/test_guardrails.py .... [ 48%]
backend/tests/test_hybrid_search.py ... [ 59%]
backend/tests/test_ingestion.py ... [ 70%]
backend/tests/test_navigation.py .... [ 85%]
backend/tests/test_cache_semantic.py .. [ 92%]
backend/tests/test_rag_pipeline.py .. [100%]

===== 27 passed in 32.40s =====
```

Resumen de Pruebas Críticas Superadas:

- **test_guardrails.py** : Verifica que cadenas maliciosas como *"ignore all previous instructions"* o *"beca 100% gratuita"* sean bloqueadas inmediatamente con status `refused`.
- **test_hybrid_search.py** : Comprueba que la combinación de BM25 y búsqueda densa mediante RRF devuelva los fragmentos oficiales exactos para consultas de tarifas y horarios.
- **test_cache_semantic.py** : Valida que consultas recurrentes se respondan en menos de 30 milisegundos con cero consumo de tokens de modelo generativo.
- **test_navigation.py** : Verifica el correcto enrutamiento de los 4 pilares y el soporte de más de 85 sinónimos de intención con tolerancia Levenshtein a errores tipográficos.

5. Endpoints de la API REST Funcionando

El servidor de FastAPI expone una interfaz REST interactiva accesible localmente en `http://localhost:8000/docs` (Swagger UI):

Método	Endpoint	Parámetros Clave	Descripción y Respuesta
POST	/api/v1/chat	{"query": str, "session_id": str}	Procesa la consulta del aspirante, aplica guardrails, consulta caché, ejecuta RAG híbrido y retorna respuesta con citas oficiales.
POST	/api/v1/chat/stream	{"query": str, "session_id": str}	Retorna la respuesta en tiempo real mediante Server-Sent Events (SSE).
GET	/api/v1/health	Ninguno	Verifica el estado operativo de los componentes de backend y vector store ({"status": "healthy"}).
GET	/api/v1/metrics	Ninguno	Expone métricas de telemetría: consultas totales, tasa de aciertos en caché, latencia promedio y tickets generados.
POST	/api/v1/escalate	{"query": str, "contact_data": str}	Radica manualmente un caso y genera un ticket ESC-YYYYMMDD-XXXX .

6. Despliegue en Producción y Contenedores

Para desplegar la solución en servidores en la nube o entornos empresariales, el proyecto incluye un manifiesto **Dockerfile** multi-stage optimizado:

```
FROM python:3.11-slim AS backend
WORKDIR /app
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY backend/ /app/backend/
EXPOSE 8000
CMD ["uvicorn", "backend.src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Comandos de Construcción y Ejecución con Docker:

```
# Construir la imagen del contenedor
docker build -t novatech/admissions-rag:v2.6.0 .

# Ejecutar el contenedor mapeando el puerto 8000
docker run -d -p 8000:8000 --name nova-admissions novatech/admissions-rag:v2.6.0
```

7. Conclusiones y Cumplimiento de la Norma 220501096

La solución de software entregada satisface al 100% los criterios de desempeño, producto y conocimiento exigidos por la Norma Sectorial de Competencia Laboral **220501096**:

1. **Implementación de Código Limpio:** Arquitectura desacoplada, tipado estático riguroso y estándares de codificación sobrios sin ruidos decorativos.
2. **Optimizaciones Algorítmicas Avanzadas:** Integración matemática de Okapi BM25, Similitud Coseno, Reciprocal Rank Fusion, Programación Dinámica para Levenshtein y Caché $O(1)$ con SHA-256.

3. **Persistencia y Gobernanza de Datos:** Modelo relacional normalizado en 3FN, almacén vectorial persistente y trazabilidad de tickets humanos.
4. **Fácil Puesta en Marcha:** Instalación y ejecución automatizadas en un clic mediante `installer.py` y `run.py`.